Partial Design Document

Section 1 — Candidate Classes and Responsibilities

These five classes were chosen because they are central to the booking process and cannot operate in isolation. Each one relies on at least one other to carry out its responsibilities, making them a strong foundation for demonstrating collaboration in this design.

Class Name: User	
Superclass: None	**Subclasses:** Staff/Student, Admin
Responsibilities	**Collaborations**
• Stores user details (userID, name, email, role) • Stores a list of health & safety certifications the user holds • Stores booking history and current active bookings • Makes a booking request • Cancels one of their own bookings • Checks how many active bookings they currently have • Enforces their active booking limit	• ***Booking*** — to create, cancel, and view bookings • ***AccessControl*** — to provide role and certification data when permission is being checked

Class Name: Booking	
Superclass: None	**Subclasses:** None
Responsibilities	**Collaborations**
• Stores booking details (booking ID, start time, end time, date, status) • Knows which user made the booking • Knows which resource is being booked • Creates a new booking after all checks pass	• ***User*** — to know who made the booking • ***Resource*** — to know what is being booked and to retrieve its existing bookings • ***AccessControl*** — to verify the user has permission and valid certifications before confirming

• Cancels an existing booking • Modifies booking time and date • Checks for time slot conflicts against existing bookings for that resource • Returns booking status (confirmed, pending, cancelled)	

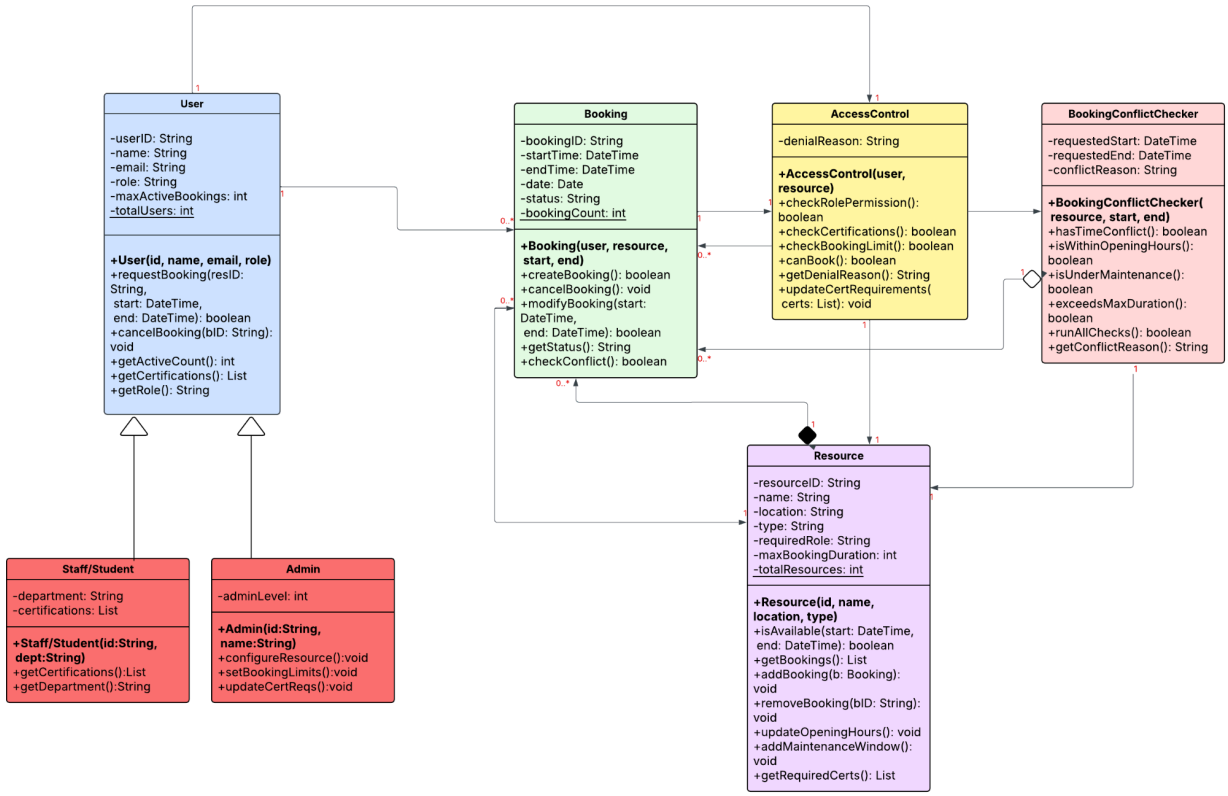
Class Name: Resource	
Superclass: None	**Subclasses:** StudyRoom, Equipment, Lab
Responsibilities	**Collaborations**
• Stores resource details (resourceID, name, location, type) • Stores required role for booking (e.g. staff only) • Stores required health & safety certifications needed to book this resource • Stores opening hours and maintenance windows • Returns whether a given time slot falls within valid available hours • Returns a list of its existing bookings • Allows administrators to update opening hours, maintenance windows, and booking limits	• ***Booking*** — to maintain a list of bookings associated with this resource • ***AccessControl*** — to provide required role and certification information when access is being checked

Class Name: AccessControl	
Superclass: None	**Subclasses:** None
Responsibilities	**Collaborations**

• Checks whether a user's role (student/staff/admin) permits them to book a specific resource • Checks whether a user holds all required health & safety certifications for a resource • Grants or denies access and returns a reason if denied • Checks whether a user has exceeded their active booking limit • Allows administrators to update certification requirements for a resource	• ***User*** — to retrieve the user's role, certifications, and current booking count • ***Resource*** — to retrieve the resource's required role and required certifications

Class Name: BookingConflictChecker	
Superclass: None	**Subclasses:** None
Responsibilities	**Collaborations**
• Checks whether a requested time slot overlaps with any existing bookings for a resource • Checks whether the requested time falls within the resource's opening hours • Checks whether the resource is under a maintenance window at the requested time • Checks whether the requested duration exceeds the resource's maximum booking limit • Returns a result (available or conflict) with a reason	• ***Booking*** — to retrieve existing bookings for a resource • ***Resource*** — to get opening hours, maintenance windows, and max booking duration

Section 2 — Class Diagrams



Collaboration Relationships

Most relationships in the design are associations, which made sense given that the majority of classes reference each other without any ownership being implied. For example, User and Booking are associated because a user keeps track of their bookings, but if a user were removed from the system, their booking records would still be meaningful. Similarly, Booking uses AccessControl temporarily to check permissions before confirming, but AccessControl is not tied to any particular booking; it simply does its job and hands control back. In most cases, navigability flows in one direction only, reflecting the fact that not every class needs to know about every other class. The exception is the relationship between Booking and Resource, which is bidirectional. A Booking needs to know which Resource it is for, and a Resource needs to maintain its list of Bookings.

The relationship between Resource and Booking is modelled as a composition because a Resource genuinely owns its bookings. If a resource were permanently removed from the system, its associated bookings would have no meaning without it and should be removed too. This felt like the most honest way to represent that dependency, with the filled diamond sitting on the Resource end and a multiplicity of 1 to 0..*.

BookingConflictChecker and Booking are modelled as an aggregation rather than a composition because the checker only borrows the bookings to inspect them — it has no claim

over them. The bookings existed before the checker was created and will continue to exist after it finishes. The hollow diamond on the BookingConflictChecker end with a multiplicity of 1 to 0..* captures this looser relationship accurately.

Five Complex Behaviour Explanations

1. Booking.createBooking(): boolean

This is the operation that actually locks in a booking once all the groundwork has been done. It starts by calling canBook() through its AccessControl association, passing in the User and Resource it already holds references to, checking the user has the right role and certifications. If that passes, it creates a BookingConflictChecker with the Resource and requested times and runs all scheduling checks. Only if everything clears does it set the status to confirmed, increment the static bookingCount, and register itself with the Resource via addBooking(). It returns false the moment any check fails. The whole thing is feasible because Booking already holds references to both User and Resource, so nothing extra needs to be fetched.

2. AccessControl.canBook(): boolean

This operation sits at the centre of the permission system. Rather than doing everything in one place, it breaks the problem into three focused checks — checkRolePermission(), checkCertifications(), and checkBookingLimit(), each of which targets a specific rule. The role check compares the user's role against what the resource requires, the certification check cross-references the user's list against the resource's requirements, and the booking limit check calls getActiveCount() to make sure they haven't hit their cap. If anything fails, denialReason is populated with a clear message, and the method returns false. This works cleanly because AccessControl holds direct associations to both User and Resource, giving it everything it needs without reaching out to any other class.

3. BookingConflictChecker.runAllChecks(): boolean

This operation ties together all the scheduling validation into a single call. It runs four checks in order, hasTimeConflict(), isWithinOpeningHours(), isUnderMaintenance(), and exceedsMaxDuration(). The time conflict check iterates through the Resource's existing bookings via getBookings(), comparing each one's start and end times against the requested slot. The opening hours and maintenance checks pull that data directly from the Resource. The duration check simply compares the requested length against the Resource's maxBookingDuration. If any check fails, conflictReason is updated, and the method stops immediately. This is feasible because the class holds a direct association to Resource and an aggregation to the Booking collection, meaning all the data it needs flows through just those two links.

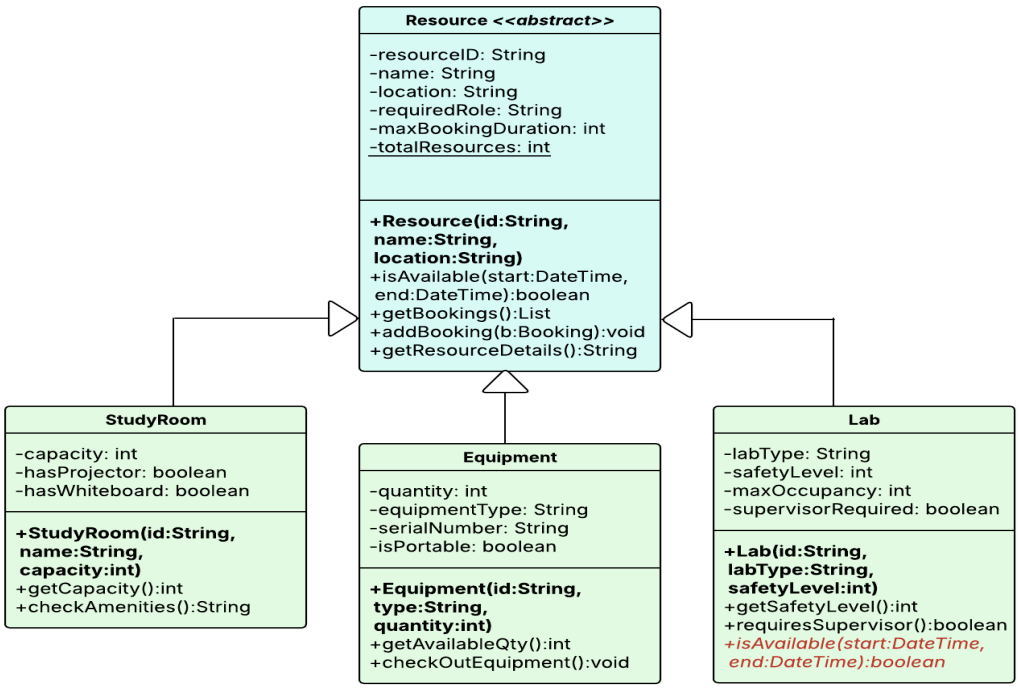
4. Resource.isAvailable(start: DateTime, end: DateTime): boolean

This operation answers a straightforward question: is this resource actually open and not under maintenance at the requested time? It checks the openingHours map first to confirm the slot falls within operating hours for that day, then scans the maintenanceWindows list for any overlap. It is worth noting that this does not check whether someone else has already booked the slot, that is BookingConflictChecker's job. This operation only concerns itself with the resource's own availability rules, and since all that data lives directly in the Resource class as attributes, no external collaborations are needed at all.

5. User.requestBooking(resID: String, start: DateTime, end: DateTime): boolean

This is the operation that kicks everything off from the user's side. The first thing it does is check its own active booking count; if the limit has been reached, it returns false straight away without going any further. Assuming that passes, it locates the right Resource, spins up an AccessControl object with itself and the Resource, and calls canBook(). If permission is granted, a new Booking is created, and createBooking() is called to complete the process, with the booking then added to the user's active list. What makes this operation particularly interesting from a design perspective is that it sets off a chain reaction involving all five classes, which is precisely why these classes were chosen and why their collaborations are designed the way they are.

Section 3 — Hierarchy Descriptions



Justification for the Resource Inheritance Hierarchy

When thinking about how to model the different types of bookable items in the system, it became clear pretty quickly that StudyRoom, Equipment, and Lab all share a lot in common. Every one of them has a resourceID, name, location, requiredRole, and maxBookingDuration, and every one of them needs to support operations like isAvailable(), getBookings(), and addBooking(). Rather than defining all of that three times over, it made far more sense to pull those shared attributes and behaviours up into an abstract Resource superclass and let the subclasses inherit them automatically. It keeps the design clean and means any future changes to shared behaviours, only need to be made in one place.

Where the subclasses differ is in the details specific to their type. StudyRoom needs to know its capacity and what facilities it has, like projectors or whiteboards. Equipment needs to track quantities, serial numbers, and whether items are portable. The lab carries extra information around safety (the safety level, maximum occupancy, and whether a supervisor is required). Lab also overrides isAvailable() from the parent class, because checking whether a lab is available involves more than just looking at opening hours; it needs to account for safety requirements that a standard room or piece of equipment simply does not have.

The other big benefit of this structure is polymorphism. Classes like Booking, AccessControl, and BookingConflictChecker all work with a Resource reference rather than having to handle StudyRoom, Equipment, and Lab, as completely separate types. This makes the system much easier to extend; adding a new resource type in the future, like a sports facility, would just mean creating a new subclass of Resource without touching anything else.